

Multi-Agent Reinforcement Learning for Safe Hovering and Navigation on Crazyflie Drones

Michael O'Sullivan, Aydin Yalcinkaya

Supervised by Professor Colin Bellinger

Part of the Machine Intelligence and Robot Learning (MIRL) Lab

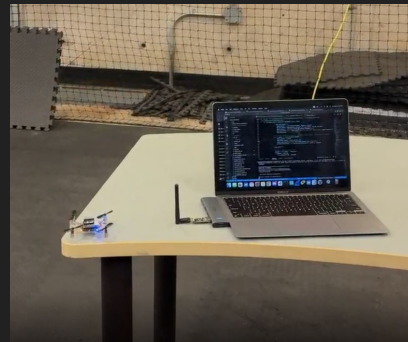
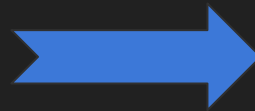
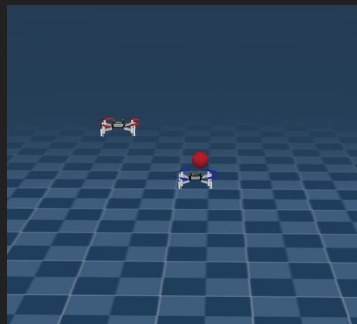
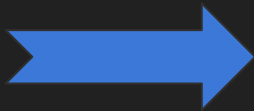
University of Ottawa

Project Overview

Goal → Apply multi-agent reinforcement learning to 2+ drones in order to achieve safe navigation and hover for a target position (red) in XYZ

Methodology

1. Set up a simulation environment for drone control (start with just 1)
2. Train agent(s) to navigate to and hover at a target in XYZ without collision
3. Test the agent(s) in simulation and on real drones



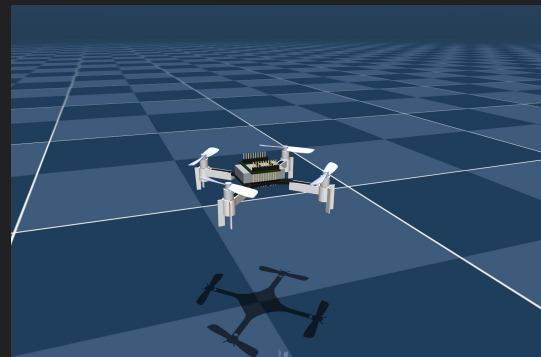
Crazyflie Drones and Simulation

Crazyflie drones

- Small, versatile, programmable drones

MuJoCo

- Open-source physics engine for robotics and more
- Provides a free physics model of the crazyflie drone
 - Drone mass, actuators...



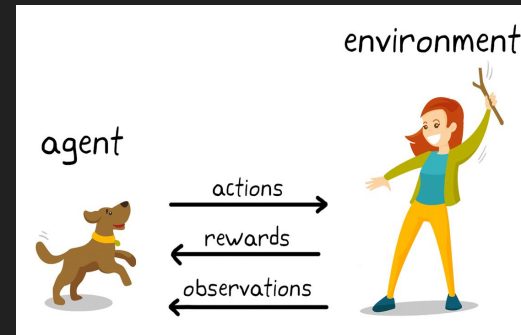
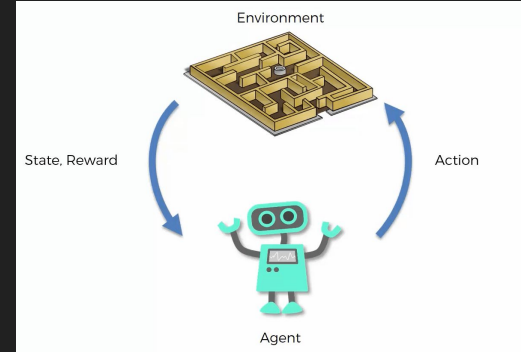
Background: Reinforcement Learning

Train an AI agent act correctly in a training environment

- Agent acts → Receives observations and rewards
- Develops a behaviour (policy) to maximize future rewards

Example: Training a dog to sit

- Dog hears “sit”, sees gesture, smells treat... → **Observation**
 - Acts randomly at first (explores behaviour)
 - Sequence of observations and actions → **Episode**
- Dog receives treat if he/she sits on command → **Reward**
 - Gradually associate “sit” with a reward → **Credit assignment**
 - Learns behaviour to maximize future rewards → **Policy**



[Robot image](#): Gordić, A. (2021). How to get started with Reinforcement Learning (RL). Medium.

[Dog image](#): (n.d.). What Is Reinforcement Learning? Www.mathworks.com.

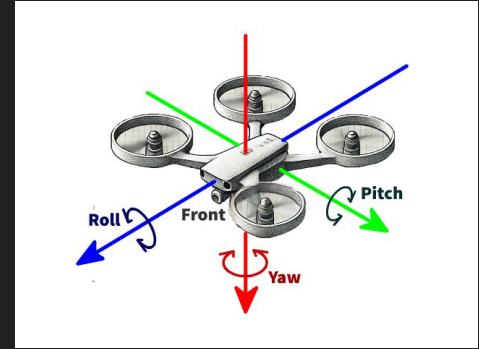
Our Environment: Choosing Observations

Per-Drone Observation

- **Standard:** Position, rotation (matrix), linear velocity, and angular velocity
- **Goal:** The relative position to target in drone body space (normalized by initial dist)
- **Multi-agent:** Relative positions of all running drones
- **Additional:** Previous action, linear velocity in drone body space

Actions

- 4 continuous actions: Drone thrust, roll, pitch, and yaw



Our Environment: Choosing Per-Step Rewards

Initial Single-Drone Baseline Hover

- Survival reward (+0.001)
- Crash/Out of bounds penalty (-1.0) → Ends environment episode

Adding Drone Goal

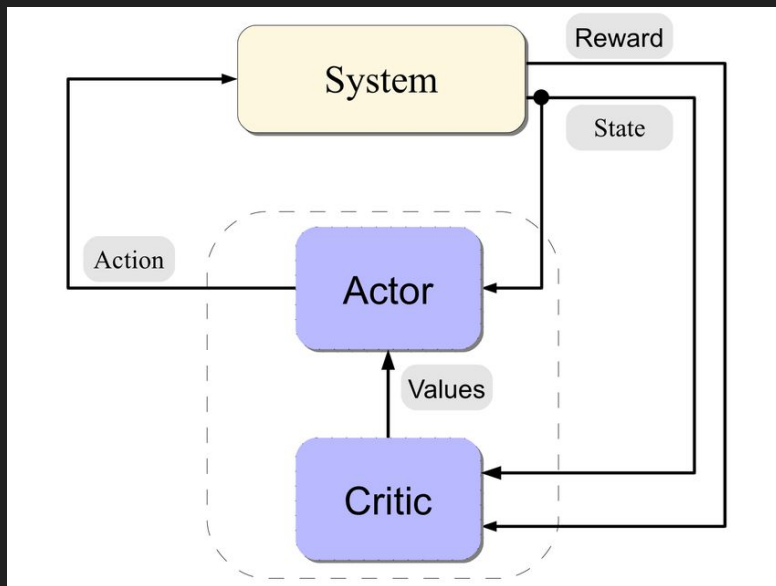
- Target distance improvement (+/-) = previous distance - current distance
- Proximity to target = +0.01 * (1 - tanh(target distance)) → (0, 0.01)

Adding Multi-Drone Considerations

- If inter-drone distance < threshold → -0.01 * (1 - distance / threshold) → [-0.01, 0]
- Drone collision penalty (-1.0) → Ends environment episode

Learning Algorithms: PPO and MAPPO

Proximal Policy Optimization (PPO) → Learn with Actor and Critic



Actor (drone controller)

- Takes action, sees state, adjusts policy based on Critic's estimate of the state's value

Critic (actor's "teacher", state value estimator)

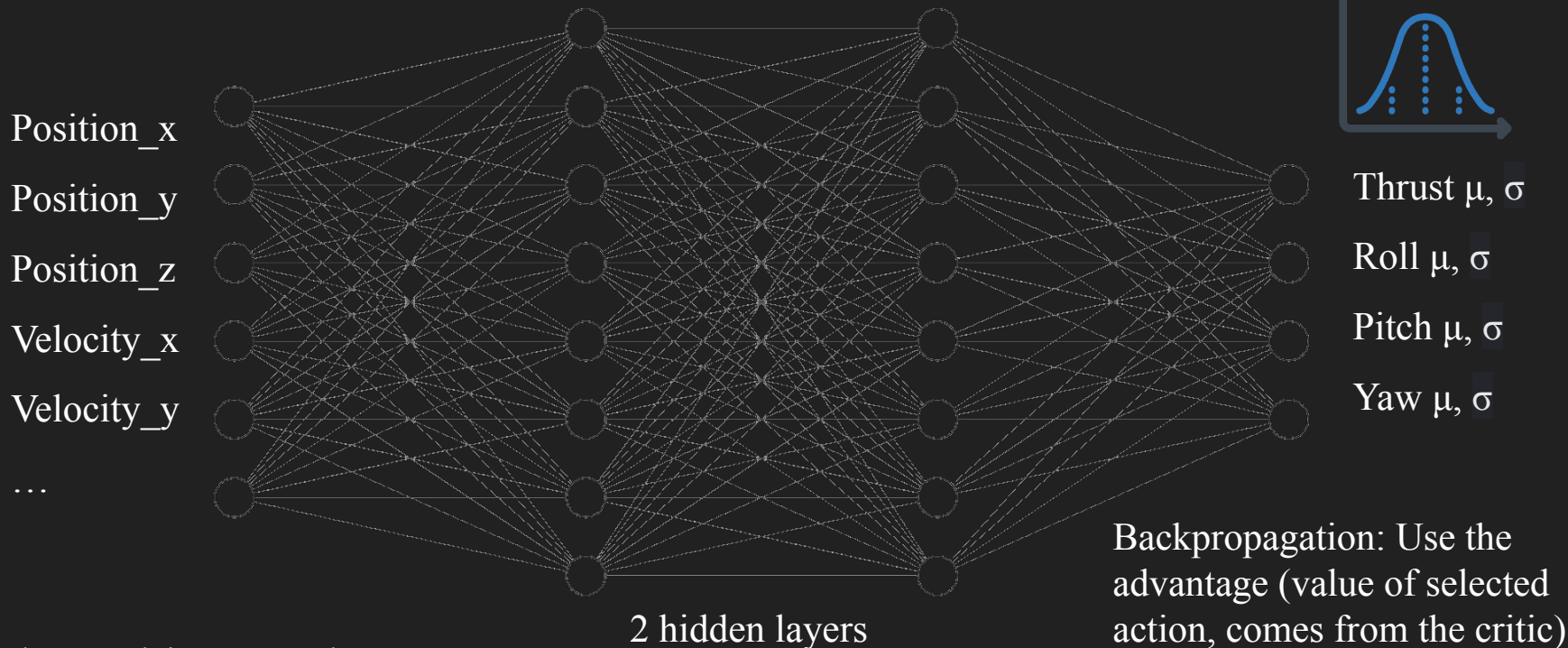
- Estimates the expected return (future reward) of the current state given the current policy

Multi-Agent PPO (MAPPO)

- 2+ actors (shared policy, per-drone obs/actions)
- Global Critic (sees all drone states, shared value)

Actor (Drone Controller)

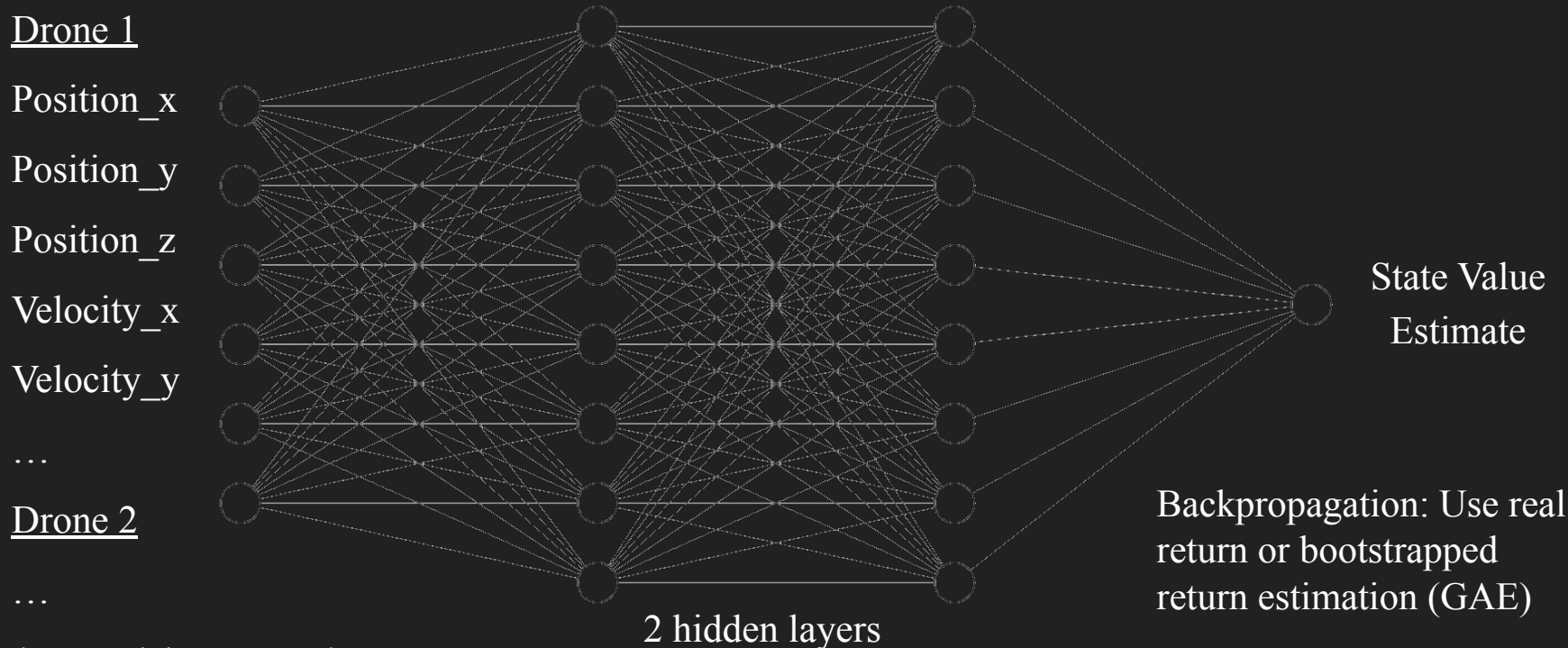
Per-drone observation $\rightarrow$ Action distributions



*Neural Network is not to scale

Critic (State Estimator)

All drone observations → Scalar value of current state based on rewards



*Neural Network is not to scale

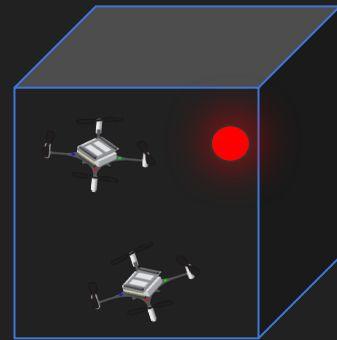
Training the agents: Methodology

Train agent (policy) in 2048 parallel environments

- Achieved with Jax, which operates on the GPU (original CPU training was slow)
- Samples each environment at each step (more sampling $\rightarrow$ faster learning)

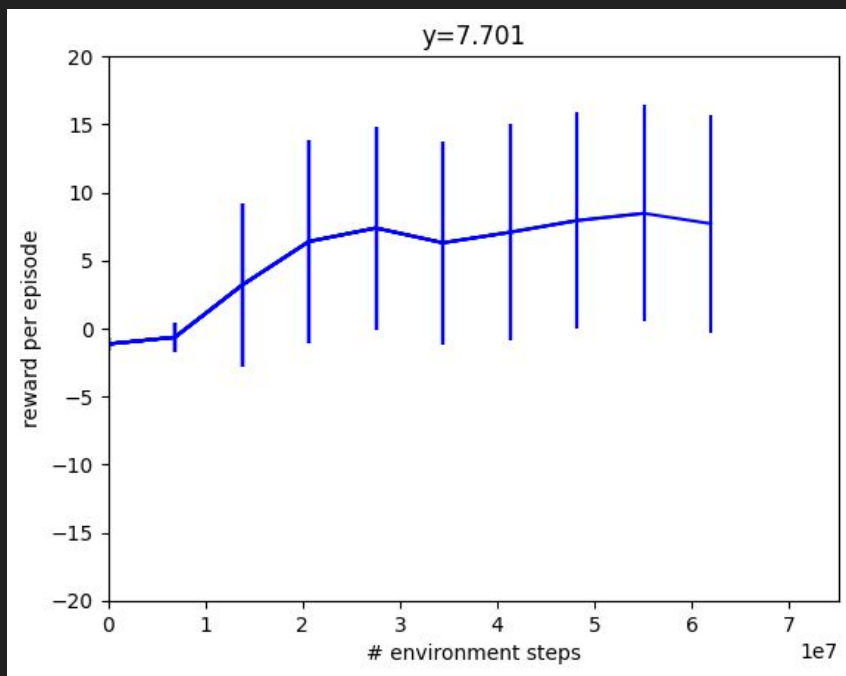
Avoid Overfitting $\rightarrow$ Random starting drone and Target state

- Goal: Explore lots of different states to better generalize behaviour
 - Drones and target within $\pm 2\text{m}$ cube, $Z > 0.1$ (+ensure drone separation)
 - Drones start with varying velocity, rotation, and angular velocity

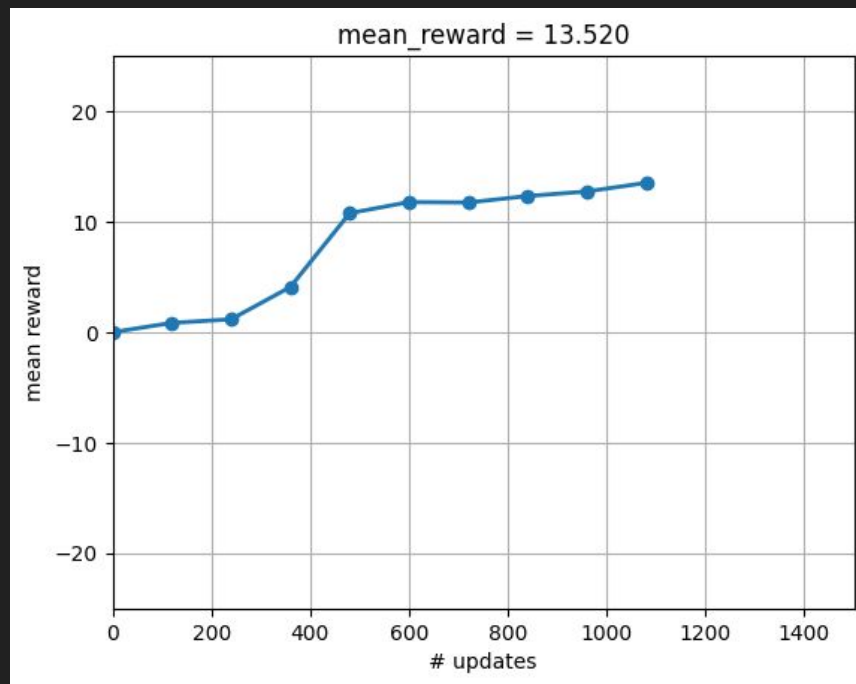


Add noise to observations and physics $\rightarrow$ Simulate hardware noise (sim2real)

Training the agents: Learning Plots



PPO training, 2048 parallel environments
60 million environment steps total, ~20 minutes



MAPPO training, 2048 parallel environments
~300 million environment steps total, ~50 minutes

MAPPO Evaluation in Sim: Test Varying Targets and Seeds

Testing each target with 64 seeds

Target #	Target (x,y,z)	Mean Return	Std Return	Mean Steps	Failures
0	[0.0, 0.0, 0.2]	5.95	0.09	703.9	64/64
1	[0.3, 0.0, 1.0]	55.13	0.04	3000	0/64
2	[-0.3, 0.0, 1.0]	49.63	0.1	3000	0/64
3	[0.0, 0.3, 1.0]	53.33	0.11	3000	0/64
4	[0.0, -0.3, 1.0]	53.62	0.09	3000	0/64
5	[0.5, 0.5, 1.2]	57.37	0.02	3000	0/64
6	[-0.5, 0.5, 1.2]	53.61	0.08	3000	0/64
7	[0.5, -0.5, 1.2]	56.82	0.6	3000	0/64
8	[-0.5, -0.5, 1.2]	52.4	0.09	3000	0/64
9	[0.0, 0.0, 2.0]	36.97	0.01	3000	0/64
10	[1.0, 0.0, 2.0]	55.53	0.07	3000	0/64
11	[-1.0, 0.0, 2.0]	21.06	0.01	3000	0/64
12	[0.0, 1.0, 2.0]	35.94	0.02	3000	0/64
13	[0.0, -1.0, 2.0]	35.19	0.01	3000	0/64
14	[2.0, 2.0, 2.0]	50.34	0.02	3000	0/64
15	[-2.0, 2.0, 2.0]	11.74	0	3000	0/64
16	[2.0, -2.0, 2.0]	52.41	0.02	3000	0/64
17	[-2.0, -2.0, 2.0]	13.64	0.01	3000	0/64
18	[0.0, 0.0, 3.0]	14.94	0.01	3000	0/64
19	[-3.0, 0.0, 3.0]	7.68	0	3000	0/64
20	[0.0, -3.0, 3.0]	12.82	0.6	2982.9	1/64
21	[3.0, 3.0, 3.0]	42.05	0.02	3000	0/64
22	[5.0, 5.0, 5.0]	19.07	0	3000	0/64
23	[7.0, 7.0, 7.0]	9.21	0.28	2555.4	64/64

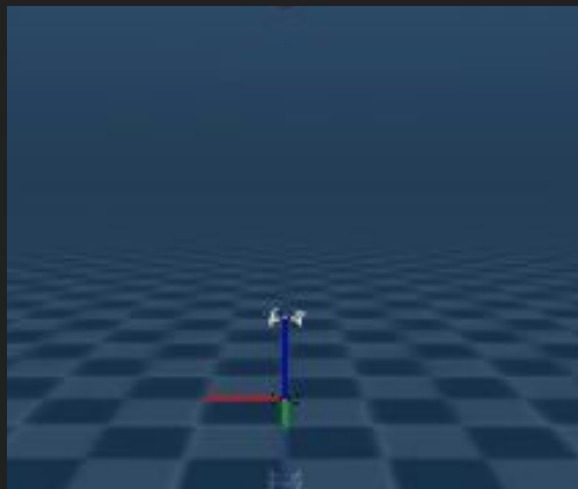
Minimum,
close to
ground

Varying
success for
mid-ranges

Maximum,
far away

PPO Visualization and Evaluation on Real Drones

With Target = (0, 0, 0.5-1)m



PPO in simulation (+noise)



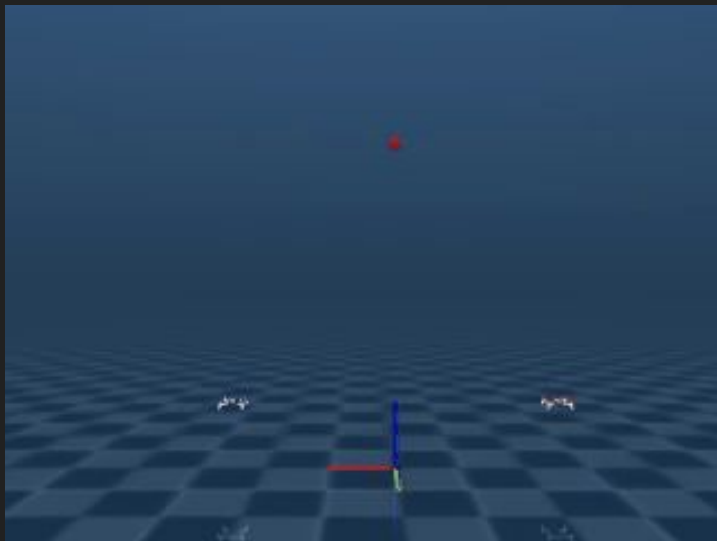
PPO on real drone
(Some drift, oscillation...)



Manual PID hover
(stability comparison)

MAPPO Visualization and Evaluation on Real Drones

With Target = (0, 0, 1)



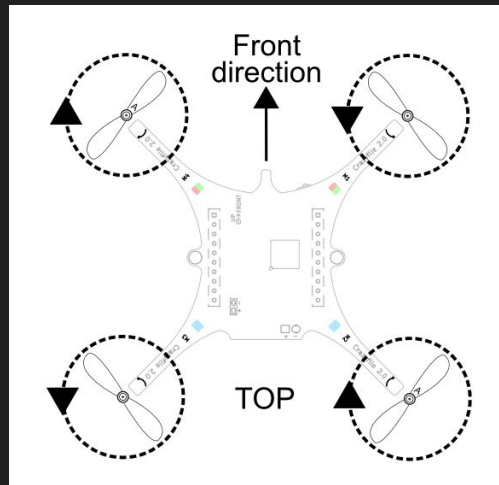
MAPPO in simulation (+noise)



MAPPO on real drones
(Promising start, but leads to crash)

Performance Difference Between Sim and Hardware (sim2real)

- Significant differences between CF2 model and real drone control mappings
 - From the CF2 MuJoCo documentation: “The *ctrlrange* limits are currently arbitrary and need to be further tuned.”
- Drone physics differences and sensor noise
 - FlowDeck & battery position
 - Battery voltage
 - Propellor orientation
 - Sensor (observation) noise



Conclusions and Future Work

Results

- Successful single and multi-agent behaviour in simulation
- Successful single agent but unsuccessful multi-agent behaviour on real drones

Improvements and Future Work

- Standardize observation input for MAPPO (maintain running mean & std)
 - Standardization is done in the PPO algorithm (may explain difference)
- Bring the MuJoCo model physics closer to actual hardware

Extensions

- Different behaviours (e.g. path following, cat and mouse), more drones...

Thank You

**We will now take
any questions**



Blooper: Crazyflie AI takes revenge against its creators